OUTAGE DISSECTIONS • INCIDENT POST-MORTEM • DATABASE RELIABILITY

GitHub Outage Dissection: Actions Breakdown & Data Inconsistency

Curated from Arpit Bhayani's Architecture Dissection Series • Incident Date: October 21, 2018

EXECUTIVE ARCHITECTURAL ABSTRACT

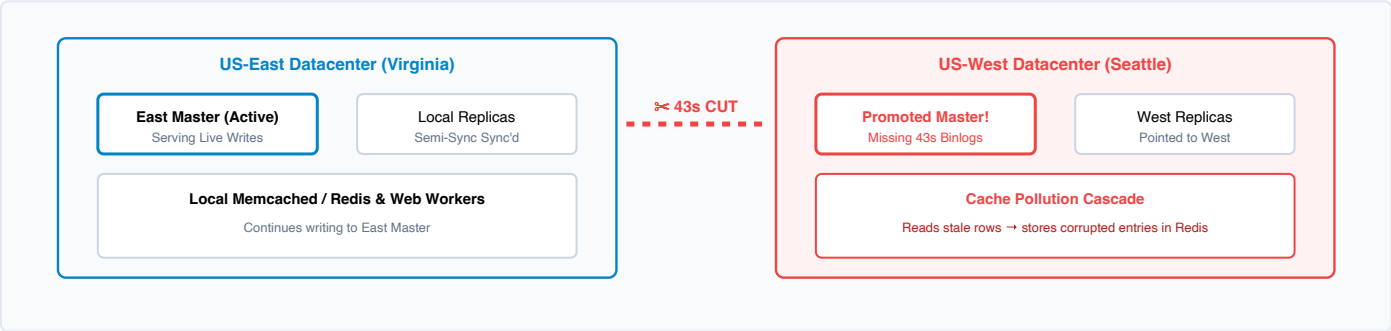
On October 21, 2018, a **43-second optical link interruption** between GitHub's primary US-East (Virginia) and secondary US-West (Seattle) data centers caused a **24-hour, 11-minute major outage**. The network severance caused automated topology managers to trigger cross-region failover, promoting lagging replicas to masters while the primary master continued serving traffic. This resulted in dual-master split-brain, divergent MySQL binlogs, and catastrophic cache pollution across Redis and Memcached.

1. THE 43-SECOND FAULT & 24-HOUR RECOVERY PARADOX

Modern distributed systems are designed to mask transient network failures. However, when failure detection triggers automated failovers across wide-area networks without verifiable fencing, brief blips produce severe state divergence:

- **Optical Backbone Disconnect:** Routine transit provider maintenance severed 100Gbps links between US-East and US-West for 43 seconds.
- **The False Death Assumption:** Orchestrator daemons in US-West observed zero heartbeats from US-East and concluded the primary datacenter was destroyed.
- **Dual-Master Emergence:** US-West promoted local read replicas to active masters. However, US-East never went down—it continued processing thousands of user commits, issue updates, and webhook deliveries concurrently.

2. TOPOLOGICAL DISSECTION: THE CROSS-REGION SPLIT-BRAIN



3. THE MECHANICS OF STATE INVERSION & CACHE POISONING

When the 43-second link re-established, GitHub faced a profound state inversion problem:

Binary Log & Auto-Increment Collision

Both MySQL clusters allocated identical auto-increment IDs to completely different entities (e.g. ID 1,000,001 was an Issue in East and a Pull Request in West). Neither cluster could replicate from the other without crashing on primary key unique constraints.

Cache Layer Poisoning (The 24h Culprit)

West Coast web workers serviced user traffic against the lagging West master. When records were missing or outdated, application read-repair logic wrote stale data into Redis and Memcached with 24-hour TTLs, persisting corruption even after network recovery.

4. IMPACT ON GITHUB ACTIONS, WEBHOOKS & GIT RPC

The split-brain database state directly incapacitated core GitHub platform primitives:

- **GitHub Actions Pipeline Stalls:** Workflows remained queued indefinitely because webhook trigger events were acknowledged by the West Coast but never persisted in the East database.
- **Pull Request Inconsistencies:** Merge buttons disappeared, review statuses diverged, and comments posted to the West master vanished from users viewing East data.
- **Webhook Dropping:** Integrations failed silently; downstream CI systems received duplicate or out-of-order event payloads.

5. THE SURGICAL RECOVERY & DATA RECONCILIATION PROTOCOL

Rather than rolling back to a snapshot (which would wipe out 24 hours of valid developer work globally), GitHub engineers executed a live data surgical reconciliation:

Step-by-Step Production Recovery Pipeline

1. **Maintenance Mode Freeze:** At 23:15 UTC, GitHub placed the entire platform into read-only maintenance mode to stop further binlog divergence.
2. **Binlog Extraction:** Extracted binary logs from the US-West master during the divergence window. Isolated transactions into SQL insert/update patches.
3. **Conflict Resolution Heuristics:** Replayed transactions into the authoritative US-East master. Where auto-increment keys collided, custom scripts assigned new IDs and updated relational foreign keys across 15+ shards.
4. **Global Cache Purge:** Flushed all Redis and Memcached instances across both datacenters to eliminate poison cache keys.
5. **Topology Restoration:** Re-imaged US-West replicas from fresh US-East snapshots and restored semi-sync replication pipelines.

6. COMPARATIVE ANALYSIS: FAILURE HANDLING MODELS

Architecture Model	Failover Behavior	Split-Brain Risk	Recovery Complexity
GitHub 2018 (Unfenced Cross-Region)	Local Orchestrator promoted replica on 43s timeout	**Extreme** (Dual active masters formed)	24h surgical binlog reconciliation
Fenced Master (Fencing Tokens)	Requires fencing acknowledgment before promotion	**Zero** (Old master blocked from disk)	Sub-minute automatic recovery
Consensus Replication (Raft/Paxos)	Requires 3-region quorum (2 of 3 regions)	**Zero** (Isolated region cannot form quorum)	Automatic leader election within 5s

7. ARCHITECTURAL TAKEAWAYS FOR DISTRIBUTED SYSTEMS

1. **Never Fail Over Across WAN Without a Third Witness:** In a 2-datacenter setup, network partitions are indistinguishable from remote datacenter death. Automated failover requires an independent third witness site to arbitrate quorums.
2. **Fencing Tokens are Vital:** A promoted master must possess a monotonically increasing epoch token. Storage engines and proxy layers must reject all requests bearing stale epochs.
3. **Decouple Cache Invalidation from Database Topologies:** Topology changes must trigger automatic cache namespace shifting (e.g., bumping cache key version prefixes) to prevent stale replica data from poisoning live clusters.